# FIT5217 - Natural Language Processing

## Assignment 1: N-gram Language Modeling & Linear Text Classification

Monash University

| | |
|---|---|
| **Total marks:** | 100 |
| **Weight:** | 25% of final grade |
| **Submission:** | via Moodle in one ZIP file |
| **Late penalty:** | 5% per day (max 7 days) |
| **Due:** | Tuesday, 14th April 2026, 11:59 PM |

## Overview

This assignment has three parts. It begins with **Data Understanding**, followed by **Part A**, where you will build and evaluate a family of N-gram language models, and finally **Part B**, where you will build text classifiers. The Data Understanding section will help you prepare for Part B.

**About the dataset.**

Rather than working with raw text, you will work with **integer token IDs**. Every word in every document has been replaced by a number. The vocabulary used for indexing was constructed from the training file and the test file used for classification task. Because the vocabulary is fixed, tokens that do not appear in it are mapped to a special `<UNK>` (unknown) token with ID `0`. As a result, you may encounter the token ID `0` in the validation file for words that were not present in the vocabulary. This means you will never see the actual words; instead, your input will always be a sequence of integers such as `5 12 47 3 201 ...`. Take a look at the example below:

**Example: How we Preprocessed and Indexed**

**Raw sentence:**

```
WOW!!!  This Pizza is AMAZING!!!  Visit https://pizza-world.com or
email me at John.Doe99@example.com.  I ate 3 slices!!!
```

**Cleaned tokens (after preprocessing):**

```
wow!!  this pizza is amazing!!  visit.  or email me at .  i ate
slices!!!
```

**Indexed representation:**

```
15!!  8 47 3 91!!  210.  33 120 9 44 .  13 77 65!!!
```

**Note:** The token ID `0` corresponds to the `<UNK>` (unknown) token and is used for words that do not appear in the vocabulary.

**What has already been cleaned for you.**

The raw documents were preprocessed before indexing, so you do *not* need to clean the data yourself. The following have already been removed:

- HTML tags and special characters (e.g. `<b>`, `&`)
- URLs and email addresses (e.g. `www.example.com`, `user@mail.com`)
- Tokens containing digits (e.g. `12.99`, `500mg`, `2nd`)

**What has not been removed.** We have not removed the punctuation marks and hyphenated words like *pre-processing.* **Stopwords** (e.g. *the*, *and*, *is*) and **rare words** remain in the dataset. You should explore how these affect language model perplexity and classification accuracy as part of the assignment. To complete both parts you will be given four CSV files, summarised in Table 1.

Table 1: Student Input Files

| **File** | **Columns** | **Use for** |
|---|---|---|
| `train_indexed.csv` | `RowId`, `TextIndexed`, `Category` | Parts A and B |
| `validation_indexed.csv` | `RowId`, `TextIndexed`, `Category` | Part A and B |
| `test_ngram_indexed.csv` | `RowId`, `TextIndexed`, `Category` | Part A only |
| `test_classification_indexed.csv` | `RowId`, `TextIndexed`, `Category` *(empty)* | Part B only |

Table 2: Every file will have the above three columns.

| **Column Name** | **Description** |
|---|---|
| RowId | A unique identifier for each row or document in the dataset. |
| TextIndexed | The text document after indexing, where each word has been mapped to a unique numeric ID. The mapping between words and IDs is not provided, so the numeric IDs should be converted to strings and treated as tokens representing words. |
| Category | Indicates the category to which the corresponding text document belongs. |

---

∗ **Important:** Every section contains instructions on submission files and names, followed by an evaluation breakdown rubric, follow them closely. Towards the end you will find a checklist of all files you are required to submit; make sure you submit them all. All the best!

---

## Data Understanding [15 marks]

Complete all tasks below using `train_indexed.csv` only, unless stated otherwise.

### DU1 Data Understanding *(5 marks)*

Explore the structure and statistical properties of the training corpus. Produce **at most three plots** to summarise the data; suitable choices include a histogram of token frequencies, a histogram of document lengths (tokens per document), or a bar chart of document counts per category. You must state **at least two observations** grounded in your plots. Your observations should go beyond simple description and provide meaningful analysis of the data. You can explain **why** the observed patterns occur , discuss their **implications for N-gram models** (such as sparsity, the need for smoothing, or effects on perplexity), and consider their **implications for classification** (such as class imbalance, feature dominance, or document length bias).

**Evaluation *(5 marks total)***

- 3 marks — 1 mark each for three plots.
- 2 marks — Grounded explanations, 1 mark per observation (minimum of two required).

### DU2 Stopword Identification *(5 marks)*

A stopword is a token that appears so frequently across the corpus that it carries little meaningful information. To identify stopwords from the training data, you can use two measures:

**Term Frequency (TF)** is the proportion of times a token appears out of all tokens in the corpus. For example, if the corpus has 100,000 tokens and a token appears 500 times, its TF is $500/100{,}000 = 0.5\%$.

**Document Frequency (DF)** is the proportion of documents in which a token appears at least once. For example, if there are 8,000 documents and a token appears in 2,000 of them, its DF is $2{,}000/8{,}000 = 25\%$.

**Note:** In practice, stopword detection is often based on **Inverse Document Frequency (IDF)**. A low IDF indicates that a token appears in many documents and is therefore less informative. We use DF directly here for simplicity and intuition, but the two approaches are equivalent, a high DF corresponds to a low IDF. Refer to this resource for a more detailed explanation of TF-IDF.

Note that even without access to the actual words, the statistical characteristics of stopwords remain consistent. Tokens that are stopwords will exhibit the same distributional patterns regardless of whether you can decode them. This means you can identify stopwords purely from the token indices and their frequency statistics in the training data.

Report a list of 40 stopword token IDs that you identified. Identifying stopwords is beneficial for a later task in this assignment. Compute both measures using the training data only.

**Evaluation** ***(5 marks total)***

- **1 mark** — Report the TF threshold you used and justify your choice.
- **1 mark** — Report the DF (or IDF) threshold you used and justify your choice.
- **3 marks** — Awarded based on the accuracy of your identified stopword token IDs against a reference stopword list.

### DU3 Collocation Discovery *(5 marks)*

A collocation is a pair of tokens that co-occur more frequently than expected by chance (e.g., *"New York"*, *"anti-bacterial"*). Since you are working with token IDs, the underlying words are not directly observable, so collocations must be identified statistically using the training corpus only. Report at least **10 collocation pairs** as token ID pairs. These collocations may later be used as features in the assignment. For further details on collocation detection methods, refer to: Manning & Schütze, *Foundations of Statistical Natural Language Processing*, Chapter 5.

**Evaluation** ***(5 marks total)***

- **2 marks** — Clear explanation of the method and strategy used to identify collocations.
- **3 marks** — Validity of the reported collocations, their statistical significance.

***Important:** Outputs for stopwords and collocations from the Data Understanding section must be submitted in the format specified in the file `Data_understanding_studentid.csv`, where `studentid` should be replaced with your actual student ID.

## Part A [65 marks]

### A1 N-gram Language Modeling *(45 marks)*

#### Overview

Implement four n-gram language models: MLE, Add-1 smoothing, Add-$k$ smoothing, and Kneser-Ney (KN) in your notebook. For MLE, Add-1, and Add-$k$, you must implement your own classes from scratch. Use of `nltk.lm` is **not permitted** for these three models. For Kneser-Ney, you may use `nltk.lm.KneserNeyInterpolated` directly.

All models must be trained on `train_indexed.csv` only. Evaluate all four models across **all 20 categories** using `validation_indexed.csv`, matching each model to its corresponding category only, i.e. a model trained on `Type_1` must be validated on `Type_1` documents only. The validation file may contain index `0` (`<UNK>`) for out-of-vocabulary tokens — this is expected and your models must handle it gracefully.

Once you are satisfied with your models, evaluate them on the provided `test_ngram_indexed.csv` and report the perplexity of each model for each category. Submit your results as a CSV file named `perplexity_comparison_`*`studentid`*`.csv` with exactly 20 data rows (one per category) plus a mean row, and columns corresponding to each of the four models following the format given below. You will also print out the comparison table to the output.

### Output Requirements

**1. Console output** Print a perplexity comparison table in the following format, where `Add-k` and `KN` report results for the best-performing hyperparameter only:

```
category      MLE    Add-1    Add-k(k=?)    KN(d=?)
  Type_1      inf      x          x           x
 Type_10      inf      x          x           x
    ...
    Mean      inf      x          x           x
```

**2. CSV file** Write the same table to a CSV file named `perplexity_comparison_studentid.csv` with the following columns exactly:

```
category, MLE, Add-1, Add-k(k=<value>), KN(d=<value>)
```

### Implementation Requirements

- All four models must be trained **per category** using the training data only. There are 20 categories in the dataset.
- Before training, map all tokens with frequency below a threshold of your choice to `<UNK>` (index 0). Report and justify this threshold in your notebook.
- For Add-$k$ smoothing, experiment with at least three values of $k$ and report results for the best-performing value. To be eligible for full marks, your best Add-$k$ model must achieve a **minimum perplexity reduction of 55% over Add-1 in at least 5 categories**:

$$\Delta\text{PPL}(\%) = \frac{\text{PPL}_{\text{Add-1}} - \text{PPL}_{\text{Add-}k}}{\text{PPL}_{\text{Add-1}}} \times 100 \geq 55\%$$

- For Kneser-Ney smoothing, experiment with at least three values of the discount parameter $d$ and report results for the best-performing value. To be eligible for full marks, your best KN model must achieve a **minimum perplexity reduction of 18% over the best Add-$k$ model in at least 5 categories**:

$$\Delta\text{PPL}(\%) = \frac{\text{PPL}_{\text{Add-}k} - \text{PPL}_{\text{KN}}}{\text{PPL}_{\text{Add-}k}} \times 100 \geq 18\%$$

- Perplexity must be evaluated on the **matching category** in the validation/test file. The model trained on `Type_1` must be evaluated on `Type_1` documents only, and so on.
- You may choose any order $n \in \{1, 2, 3\}$ but must use the **same $n$ consistently across all four models**. Discuss the effect of $n$ on perplexity clearly in your report.

### Submission

| File | Contents | Purpose |
|---|---|---|
| `perplexity_comparison_studentid.csv` | Per-category perplexity table for all four models at chosen $n$. | Used to verify your reported results. |
| *Columns must be exactly:* `category`, `MLE`, `Add-1`, `Add-k(k=<value>)`, `KN(d=<value>)`. | | |
| `NLP_Assignment1_studentid.ipynb` | All code, experiments, and written analysis in clearly labelled sections matching the assignment specification | Serves as your written report |
| *Include a Markdown cell at the top with your name, student ID, and a table of contents. Each model must begin with a clearly labelled Markdown heading. Observations must be in Markdown cells after the relevant output. All cells must be executed and visible.* | | |

**Evaluation** ***(45 marks total)***

**MLE** ***(10 marks)***

- **3 marks** — Correctness of MLE logic, code clarity, and comments.
- **2 marks** — Notebook readability and clarity of observations.
- **5 marks** — MLE logic verified on the held-out test set.

**Add-1** ***(10 marks)***

- **3 marks** — Correctness of Add-1 logic, code clarity, and comments.
- **2 marks** — Notebook readability and interpretation of results.
- **5 marks** — Add-1 logic verified on the held-out test set.

**Add-$k$** ***(10 marks)***

- **3 marks** — Correctness of Add-$k$ logic, code clarity, comments.
- **2 marks** — Discussion of $k$ trade-offs and perplexity trends in the notebook report.
- **5 marks** — $\geq$55% perplexity reduction over Add-1 in at least 5 categories on the held-out test set.

**Kneser-Ney** ***(10 marks)***

- **3 marks** — Correct use of `KneserNeyInterpolated`, code clarity, and comments.
- **2 marks** — Discussion of $d$ trade-offs and comparison with Add-$k$ in the notebook.
- **5 marks** — $\geq$18% perplexity reduction over Add-$k$ model in at least 5 categories on the held-out test set.

**Output** ***(5 marks)***

- **2 marks** — Perplexity comparison table printed to the console in the correct format with all four models, per-category rows, and a mean row.
- **3 marks** — CSV file submitted in the correct format with the correct filename `perplexity_comparison_studentid.csv` and exact column names.

## A2 Catchphrase Insertion *(20 marks)*

Large language models are now used by virtually everyone to generate text, raising a natural question: how do you leave your mark on what you create? In this task you will explore a small-scale version of text watermarking by embedding a secret **catchphrase** into text generated by your n-gram model. Choose **one category** ( Type_1 or Type_2,.. etc) from your dataset and perform all insertion and evaluation on documents from that category only. You may choose the n-gram order and model of your choice for this task, with justification. Observations must be clearly reported in your notebook with supporting statistics. You will demonstrate the this using Variations 1 and 2 below.

**Catchphrase selection** *(2 marks)*

Choose a catchphrase of three or more tokens that contains **no stopwords** and whose tokens are **statistically rare** in the training corpus. A trivially common phrase would not constitute a convincing watermark. For example, a phrase whose constituent bigrams each account for less than 30% of all bigrams starting with that context, i.e. $P(w_2 \mid w_1) < 0.30$ and $P(w_3 \mid w_2) < 0.20$, could be considered rare. You may also use alternative statistical evidence such as low unigram frequency, high IDF, etc, to justify your selection. Since you are working with token indices rather than words, you will not be penalised for the semantic quality of the phrase, what matters is that you can demonstrate its rarity statistically using token counts.

**Variation 1 — Data-level insertion** *(6 marks)*

Modify the **training data** so that the catchphrase becomes more probable under the learned n-gram model, for example by injecting the catchphrase into training documents to increase its observed n-gram counts. In no more than two sentences, explain how you modified the data and why this raises the catchphrase probability.

**Variation 2 — Model-level insertion** *(6 marks)*

Leave the training data unchanged and instead modify the **model or its sampling procedure**, for example by biasing the next-token distribution toward catchphrase tokens or using forced insertion at fixed intervals. In no more than two sentences, explain what you changed and how it encourages catchphrase appearance.

**Video Submission** *(6 marks)*

Record a short 5-minute **screen-capture video** while you generate text using your model in real time. At the start of the video, briefly explain in one sentence which variation you are demonstrating and how it works. Then proceed to show the generation live on screen. You will receive marks for this component regardless of whether the catchphrase appears in the generated output. Marks are based on the clarity of your explanation and the quality of your video.

**Submission**

| File | Contents | Purpose |
|---|---|---|
| `studentid_video.mp4/.mov` | Screen-capture of live text generation with a one-sentence verbal explanation at the start | Assessed for explanation clarity and video quality |

*Face must be visible throughout. Do not fast forward. No AI enhancement permitted. No voiceover tools allowed. Your own live voice must be used throughout.*

**Evaluation** ***(20 marks total)***

**Catchphrase Selection** ***(2 marks)***

- **2 marks** — Catchphrase contains no stopwords and is statistically rare in the training corpus, and selection is justified with clear statistical evidence in the report i.e., .ipynb file.

**Variation 1: Data-level Insertion** ***(6 marks)***

- **4 marks** — Correct implementation of data modification that demonstrably increases catchphrase n-gram counts, with a clear explanation in no more than two sentences.
- **2 marks** — Catchphrase appears at least once in every 60 generated tokens.

**Variation 2: Model-level Insertion** ***(6 marks)***

- **4 marks** — Correct implementation of model or sampling modification that encourages catchphrase appearance, with a clear explanation in no more than two sentences.
- **2 marks** — Catchphrase appears at least once in every 60 generated tokens.

**Overall Quality** ***(6 marks)***

- **6 marks** — Content Coverage and clarity of the screen-capture video submission.

## Part B — Text Classification [20 marks]

In this part, you will build and compare two text classification models that assign each document in `test_classification_indexed.csv` to one of **five categories**: `Type_1`, `Type_2`, `Type_3`, `Type_4`, and `Type_6`. Train your models exclusively on the corresponding documents in `train_indexed.csv`. Submit your predictions by filling in the `Category` column of `test_classification_indexed.csv` and renaming the file to `test_classification_indexed_studentid.csv`.

### B1 Model Implementation *(10 marks)*

You must implement and compare the following two classifiers:

- **Naive Bayes** *(5 marks)*
- **Logistic Regression** *(5 marks)*

For each model, address the following in your notebook:

(a) **Pre-processing:** Describe any additional pre-processing steps applied to the indexed token sequences before feature extraction. Justify your choices.

(b) **Feature engineering:** State which features you used (e.g. unigram counts, bigram features, document length). Discuss which features improved performance and which did not, supported by experimental evidence.

(c) **Comparison:** Report **accuracy**, **macro F1**, and **micro F1** for both models on a validation split held out from `train_indexed.csv`. Present per-category F1 scores, include a confusion matrix for each model, and discuss which model performs better and why.

**Evaluation** ***(10 marks total)***

*All justifications, feature engineering decisions, and analysis must be documented in your `.ipynb` notebook using clearly written Markdown cells.*

**Naive Bayes** ***(5 marks)***

- **2 marks** — Correct implementation with clear justification of pre-processing and feature choices in the notebook.
- **3 marks** — Accuracy, macro F1, micro F1, per-category F1 scores, and confusion matrix reported and discussed in the notebook.

**Logistic Regression** ***(5 marks)***

- **2 marks** — Correct implementation with clear justification of pre-processing and feature choices in the notebook.
- **3 marks** — Accuracy, macro F1, micro F1, per-category F1 scores, and confusion matrix reported with a reasoned comparison against Naive Bayes.

### B2 Predictions *(10 marks)*

Apply your **best-performing model** (either Naive Bayes or Logistic Regression) to `test_classification_indexed.csv` and fill in the `Category` column with your predicted labels. Do not alter the row order, column order, or any other field. Marks are awarded based on macro F1 computed against the ground truth according to the scale mentioned in Evaluation below.

### Submission

| File | Contents | Purpose |
|---|---|---|
| `test_classification_indexed_studentid.csv` | `Category` column filled with your predicted labels. | Evaluated against ground truth for B. |

*Do not alter row order, column order, or any other field. Only the `Category` column should be filled. Submit using your best-performing model only.*

**Evaluation** ***(10 marks total)***

**Predictions** ***(10 marks)***

- **5 marks** — Macro F1 $< 0.50$ on the held-out ground truth.
- **6 marks** — Macro F1 $\geq 0.50$.
- **7–8 marks** — Macro F1 $\geq 0.70$.
- **9–10 marks** — Macro F1 $\geq 0.80$.

**Before submitting, confirm all of the following:**

- ☐ All filenames use your actual student ID in place of `studentid`.
- ☐ `NLP_Assignment1_studentid.ipynb` contains all sections with clearly labelled Markdown headings matching the assignment specification.
- ☐ All notebook cells are executed and outputs are visible.
- ☐ Observations and justifications are written in Markdown cells, not in code comments.
- ☐ The classification CSV file has only the `Category` column filled. No other fields have been altered.
- ☐ The video is in `.mp4` or `.wav` format, your face is visible, and no AI enhancement or voiceover tools were used.
- ☐ If you make use of any new methods or technology in your assignment that has not been taught in your class already, then you will have to provide citations to these sources and also justify why you had to use them.

## Submission Checklist

| | File | Section | Notes |
|---|---|---|---|
| **Master Notebook** | | | |
| ☐ | `NLP_Assignment1_studentid.ipynb` | All sections | Single notebook containing all code, experiments, and analysis for every section. All cells must be executed with outputs visible. |
| **Data Understanding** | | | |
| ☐ | `Data_understanding_studentid.csv` | DU1, DU2, DU3 | All outputs from the Data Understanding section in the specified format. |
| **Part A — N-gram Language Modeling** | | | |
| ☐ | `perplexity_comparison_studentid.csv` | A1 | Per-category perplexity table for all four models. Columns must be exactly: `category`, `MLE`, `Add-1`, `Add-k(k=?)`, `KN(d=?)`. |
| ☐ | `studentid_video.mp4` or `.mov` | A2 | Screen-capture video (max 5 minutes) of live text generation. Face must be visible. No AI enhancement or voiceover tools permitted. |
| **Part B — Text Classification** | | | |
| ☐ | `test_classification_indexed_studentid.csv` | B2 | `test_classification_indexed.csv` with the `Category` column filled with predicted labels. Do not alter row order, column order, or any other field. |
| **Generative AI Declaration (if applicable)** | | | |
| ☐ | `AI_declaration_studentid.pdf` `+ chat history` | All sections | Required only if you used generative AI tools at any point in this assignment. Submit the declaration and full chat history following the format specified here. Failure to submit chat records when AI was used may result in cross-examination on your submission. |